

```
In [1]: # Step 1: Import Libraries
import nltk
import re
import math
import pandas as pd

from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [2]: # Step 2: Download Required NLTK Data
nltk.download('punkt')
nltk.download('punkt_tab') # important
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]   C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
```

```
Out[2]: True
```

```
In [3]: text = "Tokenization is the first step in text analytics."

print("Sentence Tokenization:")
print(sent_tokenize(text))

print("\nWord Tokenization:")
print(word_tokenize(text))
```

Sentence Tokenization:

```
['Tokenization is the first step in text analytics.']
```

Word Tokenization:

```
['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'analytics', '.']
```

```
In [4]: text = "How to remove stop words with NLTK library in Python?"

stop_words = set(stopwords.words("english"))
```

```

text = re.sub('[^a-zA-Z]', ' ', text)
tokens = word_tokenize(text.lower())

filtered = [w for w in tokens if w not in stop_words]

print("\nAfter Stopword Removal:")
print(filtered)

```

After Stopword Removal:

['remove', 'stop', 'words', 'nltk', 'library', 'python']

```

In [5]: ps = PorterStemmer()

words = ["studies", "studying", "cries", "cry"]

print("\nStemming:")
for w in words:
    print(ps.stem(w))

```

Stemming:

studi
studi
cri
cri

```

In [6]: lemmatizer = WordNetLemmatizer()

print("\nLemmatization:")
for w in words:
    print(lemmatizer.lemmatize(w))

```

Lemmatization:

study
studying
cry
cry

```

In [7]: sentence = "The pink sweater fit her perfectly"

tokens = word_tokenize(sentence)

print("\nPOS Tagging:")
print(pos_tag(tokens))

```

POS Tagging:

[('The', 'DT'), ('pink', 'NN'), ('sweater', 'NN'), ('fit', 'VBP'), ('her', 'PRP\$'), ('perfectly', 'RB')]

```

In [8]: docs = [
    "Jupiter is the largest planet",
    "Mars is the fourth planet from the sun"
]

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(docs)

```

```
print("\nFeature Names:")
print(vectorizer.get_feature_names_out())

print("\nTF-IDF Matrix:")
print(X.toarray())
```

Feature Names:

['fourth' 'from' 'is' 'jupiter' 'largest' 'mars' 'planet' 'sun' 'the']

TF-IDF Matrix:

```
[[0.          0.          0.37930349 0.53309782 0.53309782 0.
  0.37930349 0.          0.37930349]
 [0.37695709 0.37695709 0.26820807 0.          0.          0.37695709
  0.26820807 0.37695709 0.53641614]]
```

In []: